

GNH Business OS, ta propre plateforme, pour le cœur du système.

Notre objectif

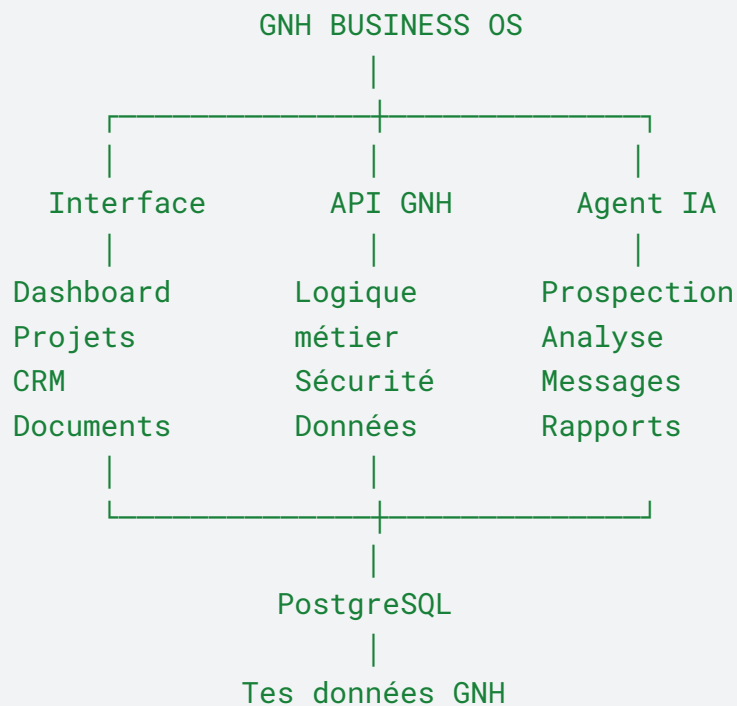
Créer une plateforme où tu peux gérer :

-  **Tableau de bord**
-  **Projets**
-  **Tâches / Kanban**
-  **CRM / prospects**
-  **Documents**
-  **Objectifs**
-  **Rapports**
-  **Agent IA GNH**
-  **Automatisations**
-  **Connexions externes** quand on en a besoin

Et surtout : **tes données et ta logique métier restent dans GNH Business OS.**

Architecture de départ

None



Et ensuite :

None

GNH BUSINESS OS

```
|
|— Google Drive      (optionnel)
|— Gmail             (optionnel)
|— Make              (optionnel)
|— Slack             (optionnel)
|— autres services   (optionnel)
```

Donc **Make** devient un connecteur éventuel, pas le cerveau de ton système.

V1 que je te propose de construire

On ne va pas essayer de tout faire d'un coup.

V1 =

1. Connexion utilisateur
2. Dashboard
3. Création de projets
4. Création de tâches
5. Vue Kanban
6. Agent IA GNH
7. Génération automatique d'un rapport
8. Base de données
9. Architecture prête pour le CRM et les automatisations

Et ton ancien projet **GNH Business AI** devient le moteur IA de cette nouvelle plateforme.

Première étape

On va donc commencer par **la structure technique complète du projet**, puis je pourrai te donner les fichiers **un par un**, comme on avait commencé pour GNH Business AI.

Je te propose ce nom de travail :

GNH Business OS — V1

avec :

None

```
gnh-business-os/  
├─ apps/  
│   ├─ web/  
│   └─ api/  
├─ packages/  
│   ├─ database/  
│   ├─ ai/  
│   └─ shared/  
├─ prisma/  
├─ .env  
├─ package.json  
└─ README.md
```

On peut maintenant passer directement à l'étape 1 : créer le squelette complet de GNH Business OS V1 et commencer le code.

Parfait. On commence **GNH Business OS V1**. 🚀

Je vais te faire avancer **étape par étape**, sans te noyer dans le code.

Étape 1 — Le socle

On va construire d'abord :

GNH Business OS

- 🏠 Dashboard
- 📁 Projets
- ✅ Tâches
- 👤 CRM
- 📄 Documents
- 🎯 Objectifs
- 📊 Rapports
- 🤖 GNH AI
- ⚙️ Automatisations

Puis on connectera ton ancien **GNH Business AI** au système.

Architecture

None

GNH Business OS



Point important : on ne va pas reconstruire 15 services immédiatement. On crée d'abord un **MVP fonctionnel**, puis on ajoute les briques une par une.

La prochaine étape est donc de créer **le projet et les premiers fichiers du backend + frontend**.

Parfait. On attaque concrètement. 🔥

Pour éviter de construire quelque chose qu'il faudra refaire ensuite, je te propose de commencer par **le socle de données de GNH Business OS** : c'est ce qui permettra ensuite au Dashboard, au CRM, aux projets et à l'agent IA de travailler sur les mêmes informations.

GNH Business OS — V1

None

```
gnh-business-os/  
|  
├─ apps/  
|   ├── web/           ← interface utilisateur  
|   └─ api/           ← serveur / API  
|  
├─ packages/  
|   ├── database/      ← base de données  
|   ├── ai/            ← GNH AI  
|   └─ shared/         ← éléments communs  
|  
├─ prisma/  
|   └─ schema.prisma  
|  
├─ .env  
├─ package.json  
└─ README.md
```

Notre première base de données

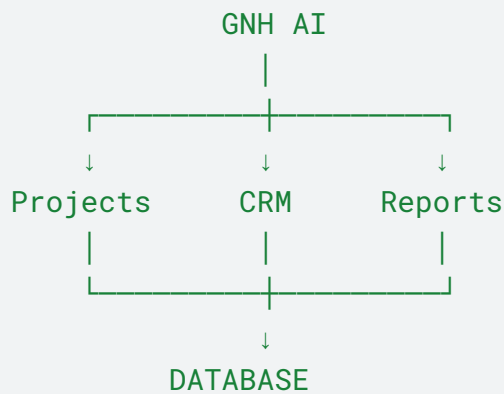
On prévoit dès maintenant les principales briques :

None

```
USER  
└─ WORKSPACE  
    ├── PROJECT  
    │   └─ TASK  
    │  
    ├── CONTACT  
    │   └─ COMPANY  
    │  
    ├── DOCUMENT  
    │  
    ├── GOAL  
    │  
    ├── REPORT  
    │  
    └─ AUTOMATION
```

Et surtout :

None



Ainsi, **l'agent IA ne sera pas une IA isolée** : il pourra travailler directement avec les données de ta plateforme.

Par exemple, tu pourras lui dire :

« GNH, montre-moi les projets en retard. »

ou :

« Prépare le rapport hebdomadaire de mon activité. »

ou :

« Analyse mes prospects et dis-moi lesquels doivent être relancés. »

Et plus tard :

« Lance cette automatisation tous les lundis. »

🔥 Notre principe

GNH Business OS = ton système central.

Monday, Teamwork, Make, Google Drive, etc. pourront éventuellement être connectés **autour** de GNH, mais ils ne seront pas obligatoires pour faire fonctionner ton système.

Si tu veux aller jusqu'au bout, on peut même prévoir dès maintenant une architecture qui permettra ensuite de transformer GNH Business OS en **une vraie plateforme SaaS utilisable par plusieurs entreprises et plusieurs utilisateurs.**

construise **GNH Business OS V1** comme un vrai produit, pas juste une maquette.

Phase 1 — Le socle

On va créer :

1. **Base de données**
2. **API**
3. **Interface web**
4. **Authentification**
5. **Dashboard**
6. **Projets + tâches**
7. **GNH AI**

Ensuite, on ajoutera CRM, documents, automatisations, rapports, etc.

Première décision technique

Pour le V1, je partirais sur :

- **Next.js + TypeScript** → interface + serveur web
- **PostgreSQL** → données
- **Prisma** → accès à la base
- **Tailwind CSS** → interface
- **GNH AI** → couche intelligente
- **Architecture modulaire** → pour pouvoir ajouter les futurs modules sans reconstruire le système

Comme on va utiliser des composants OpenAI pour l'agent IA, je vérifierai la documentation actuelle avant de figer cette partie technique.

Ce qu'on va obtenir

None

GNH BUSINESS OS		
Dashboard	Bonjour Hervé	
Projets	Projets actifs	12
Tâches	Tâches en cours	27
CRM	Prospects	84
Documents	Objectifs	6
Objectifs		
Rapports		
GNH AI	 Demander à GNH	
Automation		

Étape suivante : je te donne le projet V1 complet, avec l'arborescence et les premiers fichiers prêts à créer.

Parfait. Voici **le socle V1 de GNH Business OS**. On va le construire proprement, puis brancher ton ancien GNH Business AI dessus.

1. Crée le projet

Dans ton terminal :

Shell

```
npx create-next-app@latest gnh-business-os
```

Quand Next.js pose les questions, choisis :

None

```
TypeScript      → Yes
ESLint          → Yes
Tailwind CSS    → Yes
src/ directory  → Yes
App Router      → Yes
Turbopack       → Yes
Import alias    → @/*
```

Puis :

Shell

```
cd gnh-business-os
npm install prisma @prisma/client
npx prisma init
```

2. Structure du projet

Nous allons arriver à ceci :

None

```
gnh-business-os/
|
├─ prisma/
│  └─ schema.prisma
|
├─ src/
│  ├─ app/
│  │  ├─ dashboard/
│  │  │  └─ page.tsx
│  │  ├─ projects/
│  │  │  └─ page.tsx
│  │  ├─ tasks/
│  │  │  └─ page.tsx
│  │  ├─ crm/
│  │  │  └─ page.tsx
│  │  ├─ reports/
│  │  │  └─ page.tsx
│  │  ├─ ai/
│  │  │  └─ page.tsx
│  │  └─ layout.tsx
│  │     └─ page.tsx
│  └─ components/
│     ├─ Sidebar.tsx
│     ├─ Header.tsx
│     └─ StatCard.tsx
```

```
|   |   └─ ProjectCard.tsx
|   |   └─ lib/
|       └─ prisma.ts
|
├─ .env
├─ package.json
└─ README.md
```

3. Notre base de données

Ouvre :

```
None
prisma/schema.prisma
```

et mets :

```
None
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String    @id @default(cuid())
  email       String    @unique
  name        String?
  createdAt   DateTime  @default(now())

  projects   Project[]
  tasks      Task[]
}
```

```
model Project {
  id          String    @id @default(cuid())
  name        String
  description String?
  status      ProjectStatus @default(ACTIVE)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  ownerId String
  owner    User @relation(fields: [ownerId], references: [id])

  tasks Task[]
}

model Task {
  id          String    @id @default(cuid())
  title       String
  description String?
  status      TaskStatus @default(TODO)
  priority    Priority   @default(MEDIUM)

  dueDate DateTime?

  projectId String
  project    Project @relation(fields: [projectId], references: [id])

  userId String
  user     User @relation(fields: [userId], references: [id])

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

model Contact {
  id          String    @id @default(cuid())
  firstName   String
```

```
    lastName String?
    email     String?
    phone     String?
    company   String?
    status    ContactStatus @default(LEAD)

    createdAt DateTime @default(now())
}

model Goal {
    id          String    @id @default(cuid())
    title       String
    description String?
    target      Float
    current     Float     @default(0)

    createdAt DateTime @default(now())
}

model Report {
    id          String    @id @default(cuid())
    title       String
    content     String
    createdAt   DateTime @default(now())
}

enum ProjectStatus {
    ACTIVE
    COMPLETED
    ARCHIVED
}

enum TaskStatus {
    TODO
    IN_PROGRESS
    DONE
}

enum Priority {
```

```
    LOW
    MEDIUM
    HIGH
    URGENT
  }

  enum ContactStatus {
    LEAD
    PROSPECT
    CLIENT
    INACTIVE
  }
```

Puis :

```
Shell
npx prisma generate
```

4. Connexion Prisma

Crée :

```
None
src/lib/prisma.ts
```

```
TypeScript
import { PrismaClient } from "@prisma/client";

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient();
```

```
if (process.env.NODE_ENV !== "production") {  
  globalForPrisma.prisma = prisma;  
}
```

5. Notre premier Dashboard

Ouvrir :

None
`src/app/page.tsx`

et remplacer son contenu par :

```
None  
import Link from "next/link";  
  
export default function Home() {  
  return (  
    <main className="min-h-screen bg-slate-950 text-white">  
      <div className="flex min-h-screen">  
  
        <aside className="w-64 border-r border-slate-800 p-6">  
          <h1 className="text-2xl font-bold">  
            GNH  
          </h1>  
  
          <p className="mb-10 text-sm text-slate-400">  
            Business OS  
          </p>  
  
          <nav className="space-y-3">  
  
            <Link  
              href="/dashboard"  
              className="block rounded-lg p-3 hover:bg-slate-800"  
            >  
               Dashboard
```

```
</Link>

<Link
  href="/projects"
  className="block rounded-lg p-3 hover:bg-slate-800"
>
  📁 Projets
</Link>

<Link
  href="/tasks"
  className="block rounded-lg p-3 hover:bg-slate-800"
>
  ✅ Tâches
</Link>

<Link
  href="/crm"
  className="block rounded-lg p-3 hover:bg-slate-800"
>
  👤 CRM
</Link>

<Link
  href="/reports"
  className="block rounded-lg p-3 hover:bg-slate-800"
>
  📊 Rapports
</Link>

<Link
  href="/ai"
  className="block rounded-lg p-3 hover:bg-slate-800"
>
  🤖 GNH AI
</Link>

</nav>
</aside>
```

```
<section className="flex-1 p-10">

  <div className="mb-10">
    <h2 className="text-4xl font-bold">
      Bienvenue sur GNH Business OS
    </h2>

    <p className="mt-2 text-slate-400">
      Ton espace central pour piloter ton activité.
    </p>
  </div>

  <div className="grid gap-6 md:grid-cols-4">

    <div className="rounded-xl border border-slate-800
bg-slate-900 p-6">
      <p className="text-slate-400">
        Projets
      </p>
      <p className="mt-2 text-3xl font-bold">
        0
      </p>
    </div>

    <div className="rounded-xl border border-slate-800
bg-slate-900 p-6">
      <p className="text-slate-400">
        Tâches
      </p>
      <p className="mt-2 text-3xl font-bold">
        0
      </p>
    </div>

    <div className="rounded-xl border border-slate-800
bg-slate-900 p-6">
      <p className="text-slate-400">
        Prospects
```

```

    </p>
    <p className="mt-2 text-3xl font-bold">
      0
    </p>
  </div>

  <div className="rounded-xl border border-slate-800
bg-slate-900 p-6">
    <p className="text-slate-400">
      Objectifs
    </p>
    <p className="mt-2 text-3xl font-bold">
      0
    </p>
  </div>

</div>

<div className="mt-10 rounded-2xl border
border-slate-800 bg-slate-900 p-8">

  <h3 className="text-2xl font-bold">
    🤖 GNH AI
  </h3>

  <p className="mt-2 text-slate-400">
    Ton assistant intelligent pour analyser,
    organiser et automatiser ton activité.
  </p>

  <Link
    href="/ai"
    className="mt-6 inline-block rounded-lg bg-white
px-5 py-3 font-semibold text-black"
  >
    Ouvrir GNH AI
  </Link>

</div>

```

```
        </section>
      </div>
    </main>
  );
}
```

6. Lance GNH

Dans le terminal :

```
Shell
npm run dev
```

Puis ouvre :

```
None
http://localhost:3000
```

Tu devrais avoir la première interface de GNH Business OS.

Et ensuite, on passe au vrai fonctionnement

La prochaine brique sera particulièrement importante :

Dashboard → Projets → Tâches → Base de données

Puis nous brancherons :

GNH AI → lecture des projets → analyse → actions → rapports.

À partir de là, on commencera réellement à remplacer les fonctions de Monday/Teamwork par **ton propre système GNH**.

Parfait. On continue. 🔥

Maintenant, on va faire passer GNH Business OS d'une simple interface à une **application qui travaille réellement avec les données**.

Étape 2 — Module Projets

On veut pouvoir :

- créer un projet ;
- voir les projets ;
- définir son statut ;
- ajouter des tâches ;
- préparer ensuite la vue Kanban.

1. Crée la page Projets

Crée :

None

```
src/app/projects/page.tsx
```

avec :

None

```
import Link from "next/link";

const projects = [
  {
    id: 1,
    name: "GNH Business OS",
    description: "Construction de la plateforme GNH",
    status: "En cours",
    tasks: 8,
  },
  {
    id: 2,
    name: "GNH Business AI",
    description: "Agent IA et automatisations",
    status: "En cours",
    tasks: 5,
  },
];

export default function ProjectsPage() {
  return (
    <main className="min-h-screen bg-slate-950 p-10 text-white">

      <div className="mb-10 flex items-center justify-between">
```

```
<div>
  <h1 className="text-4xl font-bold">
    Projets
  </h1>

  <p className="mt-2 text-slate-400">
    Gère tous tes projets depuis GNH Business OS.
  </p>
</div>

<button className="rounded-lg bg-white px-5 py-3
font-semibold text-black">
  + Nouveau projet
</button>
</div>

<div className="grid gap-6 md:grid-cols-2 lg:grid-cols-3">

  {projects.map((project) => (
    <div
      key={project.id}
      className="rounded-2xl border border-slate-800
bg-slate-900 p-6"
    >

      <div className="mb-5 flex items-center
justify-between">

        <h2 className="text-xl font-bold">
          {project.name}
        </h2>

        <span className="rounded-full bg-green-900 px-3
py-1 text-xs text-green-300">
          {project.status}
        </span>

      </div>

    </div>
  )
  )}
```

```
    <p className="mb-6 text-sm text-slate-400">
      {project.description}
    </p>

    <div className="flex items-center justify-between">

      <span className="text-sm text-slate-400">
        {project.tasks} tâches
      </span>

      <Link
        href={` /projects/${project.id}`}
        className="rounded-lg border border-slate-700
px-4 py-2 text-sm hover:bg-slate-800"
      >
        Ouvrir
      </Link>

    </div>

  </div>
  )})
</div>

</main>
);
}
```

2. Page d'un projet

Crée :

None

`src/app/projects/[id]/page.tsx`

None

```
type ProjectPageProps = {
  params: Promise<{
    id: string;
  }>;
};

export default async function ProjectPage({
  params,
}: ProjectPageProps) {

  const { id } = await params;

  return (
    <main className="min-h-screen bg-slate-950 p-10 text-white">

      <div className="mb-10">
        <p className="text-sm text-slate-400">
          Projet #{id}
        </p>

        <h1 className="mt-2 text-4xl font-bold">
          GNH Business OS
        </h1>

        <p className="mt-2 text-slate-400">
          Gestion et développement de la plateforme.
        </p>
      </div>

      <div className="grid gap-6 md:grid-cols-3">
```

```
    <div className="rounded-xl border border-slate-800
bg-slate-900 p-6">
      <p className="text-slate-400">
        À faire
      </p>

      <p className="mt-2 text-3xl font-bold">
        4
      </p>
    </div>
```

```
    <div className="rounded-xl border border-slate-800
bg-slate-900 p-6">
      <p className="text-slate-400">
        En cours
      </p>

      <p className="mt-2 text-3xl font-bold">
        3
      </p>
    </div>
```

```
    <div className="rounded-xl border border-slate-800
bg-slate-900 p-6">
      <p className="text-slate-400">
        Terminées
      </p>

      <p className="mt-2 text-3xl font-bold">
        1
      </p>
    </div>
```

```
</div>
```

```
<section className="mt-10 rounded-2xl border
border-slate-800 bg-slate-900 p-8">
```

```
  <div className="mb-6 flex items-center justify-between">
```

```

    <h2 className="text-2xl font-bold">
      Tâches
    </h2>

    <button className="rounded-lg bg-white px-4 py-2
font-semibold text-black">
      + Ajouter une tâche
    </button>

  </div>

  <div className="space-y-3">

    {[
      "Créer la base de données",
      "Créer le Dashboard",
      "Créer le module Projets",
      "Connecter GNH AI",
    ].map((task, index) => (

      <div
        key={index}
        className="flex items-center justify-between
rounded-lg border border-slate-800 p-4"
      >

        <span>
          {task}
        </span>

        <span className="text-sm text-slate-500">
          À faire
        </span>

      </div>

    )))
  </div>

```

```
        </div>

    </section>

</main>
);
}
```

3. Mais attention : ceci est encore une maquette fonctionnelle

Les projets sont actuellement dans :

```
TypeScript
const projects = [...]
```

Donc si tu actualises la page, les données sont toujours les mêmes.

La prochaine étape consiste justement à supprimer ces données fictives.

On va connecter :

```
None
Interface
  ↓
API
  ↓
Prisma
  ↓
PostgreSQL
```

Et là, quand tu cliqueras sur :

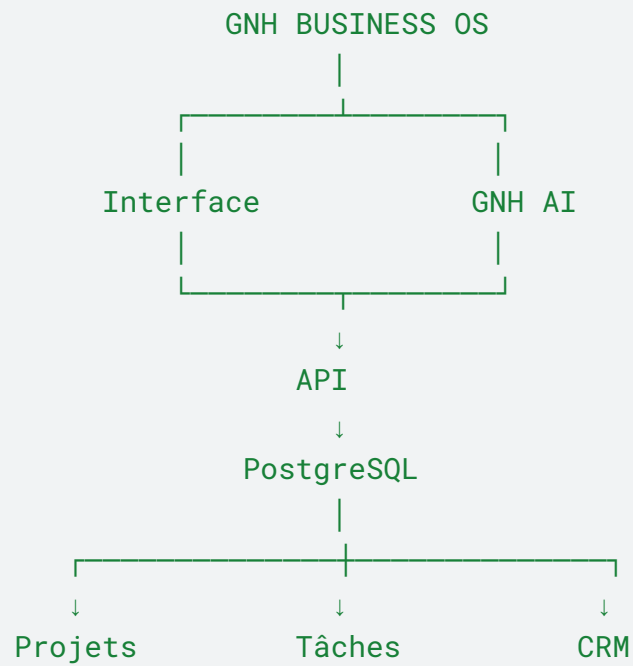
+ Nouveau projet

le projet sera réellement enregistré dans **ta base de données GNH**.

Ensuite nous ferons exactement la même chose pour les tâches.

Notre architecture finale commencera à ressembler à ça

None



Prochaine étape : on branche PostgreSQL + Prisma et on crée les premières API `/projects` et `/tasks`. C'est à ce moment que GNH commence réellement à devenir ton logiciel.